

HPPS 2024
Assignment Five
The N-body problem

Marcus Frehr Friis-Hansen lns611

Noah Wenneberg Junge qxk266

Group 7

January 5, 2025

Introduction

The code is fully functional and works as intended. We have used parallel loops to divide computation over multiple threads, and critical sections to make sure shared memory isn't accessed by multiple threads at the same time, although this slows down the computation by some margin, which isn't ideal. We are generally concerned that we haven't optimized the code as much as we could have by using more parallel loops. We have **modified** `nbody` in `nbody.c`, written `bh_mk_internal`, `bh_insert`, `bh_free`, `bh_accel`, `bh_new`, and modified `nbody` in `nbody-bh.c`. The implemented programs in these files were modified and developed using ChatGbt. During development ChatGbt was used to verify syntax, simplify and optimize. It was also used to help us understand how we could upload simulations to the MODI-system. To run the program first type `make all` in the correct directory while in `wsl`, and then follow the guide below.

```
"Generate a file containing n particles. 100000 particles are used in the
example below"
```

```
./genparticles 10000 10000_particles
```

```
"execute nbody.c for a given amount of time steps using some number of
threads. 4 threads, and 100 time steps are used below"
```

```
OMP_NUM_THREADS=4 ./nbody 10000_particles particles_out_nbody.txt
warnings_nbody.txt 100
```

```
"Execute nbody-bh.c for a given amount a time steps, using som number of
threads with theta specified. In the example below 100 time steps, 4
threads, and theta=0.5 is used"
```

```
OMP_NUM_THREADS=4 ./nbody-bh 10000_particles particles_out_nbody_bh.txt
warnings_nbody_bh.txt 100 0.5
```

We have used a the MODI system to capture the benchmarks for `nbody.c` and `nbody-bh.c` for strong and weak scaling shown in table 1,2,3 and 4. The MODI system gives access to 256 GiB of memory and two 32-core CPUs. The benchmarks shown in table 5 where performed on a Lenovo Yoga Slim 7x Snap-12Core/16/512. .

3.1 Update warnings list in `nbody.c`

The part of code that handles the particles that are too close to the spaceship (origo), is a section within the `nbody` function. It measures the distance from the current particle and checks if it is below the warning distance. If so, within a critical section, it increases the warning count and resizes the array used to store the warning, using the `realloc` function, to allocate memory for one more warning. The current step and particle index is recorded so we know when the particle was too close to the spacecraft, and which particle it was. The code is functionally correct and uses a critical section to make sure only one thread is used at a time to update the warnings. This part of the code was developed using ChatGbt.

3.2 Naive simulation in `nbody.c`

The upper part of the `nbody` function simulates the motion of `n` particles over some number of `steps`, it stores the particles in an array of particles `ps`, while also keeping a count and array of the warnings as stated in the section above. it uses parallelization to distribute the computation for the force and velocity update for each particle. Individual threads compute for separate particles, this is safe as the order for this computation is irrelevant. We do still however use a critical section to update the particles's force, as the array that stores the particles is shared memory. This part of the code was developed using ChatGbt.

1. Questions for `nbody.c`

1. a) Measure and show the weak and strong scaling of `nbody` as you vary the number of threads. Explain how you chose the datasets.

	Particles	Threads	Simulation time
Simulation 1	100 000	1	519.839069 seconds
Simulation 2	100 000	2	260.474079 seconds
Simulation 3	100 000	4	129.573882 seconds
Simulation 4	100 000	8	65.039711 seconds

Table 1: Strong scaling of `nbody.c` on the MODI System with 32-core CPUs measured in simulation time

	Particles	Threads	simulation time
Simulation 1	5000	1	12.891634 seconds
Simulation 2	10000	2	26.082444 seconds
Simulation 3	20000	4	104.747037 seconds
Simulation 4	40000	8	418.078543 seconds

Table 2: Weak scaling of `nbody.c` on the MODI System with 32-core CPUs measured in simulation time

The strong scaling of the program is almost linear, which is excellent. Every time the number of threads doubles, the execution time is almost halved. This can be seen as the number of threads increases from 1 to 2, all the way up to 8 threads. The weak scaling however, is poor. As the workload increases proportionally to the number of threads, the runtime increases exponentially. For strong scaling we chose a dataset size, we thought would give a execution time large enough to interpret on. For weak scaling we chose to scale the dataset based on 5000 particles per core. We found this size sufficient as it didn't take too long to run on the MODI system and would give us a easily interpretable response.

2. **b) Determine (in whatever way you find best) the performance overhead of updating the warning list.**

	Particles	Threads	simulation time
no warnings	100000	4	129.530507 seconds
with warnings	100000	4	131.542191 seconds

Table 3: Warning overhead comparrison on the MODI system with 32-core CPUs measured in simulation time

$$\frac{131.542191 - 129.530507}{131.542191} = 0.01604108823$$

The performance overhead of updating the warning list is $\approx 1.6\%$. The overhead being very small signifies that the code that updates the warning list has been efficiently implemented. We determined this by running the simulation with and without this implementation, and comparing the simulation times.

3. **c) Are there workloads for which updating the warning list can have a ruinous impact on parallel scalability, and why?**

Yes there are. As the amount of particles simulated increases, the likelihood that particles trigger a warning also increases. The implementation uses a critical section to increase the warning count, resize the array and record warnings to the array. The critical section ensures that only one thread may use the shared memory array at a time, this ensures a safe environment for accessing the array, it also serves as a bottleneck for the program. When multiple threads all need to access the shared memory, they must wait for their turn. As the number of particles increase the critical section will further act as a bottleneck, as the number of warning also increases, relative to the number of threads. This means that adding any more threads would have less of an impact on the runtime of the program as the number of particles grow. And that is why the critical section severely limits the programs parallel scalability. This effect can also be seen above in table 2, which shows the programs weak scalability.

4. **d) Do you have any memory leaks, and how can you know?**

To detect memory leaks in the 'nbody.c' simulation, we used the 'leaks' command on macOS, which tracks memory allocation and deallocation during the execution of the program.

Command used:

```
leaks --atExit -- ./nbody particles particles.out warnings.out 10
```

The output of the command showed that there were no memory leaks detected in the 'nbody' simulation:

```
Process 60308: 204 nodes malloced for 24 KB
Process 60308: 0 leaks for 0 total leaked bytes.
```

This indicates that the program properly freed all dynamically allocated memory, and no memory was left allocated after the program exited.

3.3 Barnes-Hut simulation in `nbody-bh.c`

The code is the finished implementation of the Barnes-Hut N-body simulation. We have tried to parallelize it as well as we could with OpenMP, but if given more time we could have optimized it further. The functions in `nbody-bh.c` were developed using ChatGbt. This is also stated in the file itself.

2. Questions for `nbody-bh.c`

1. a) Measure and show the strong scaling of `nbody-bh` as you vary the number of threads. Explain how you chose the datasets.

	Particles	Threads	Simulation time
Simulation 1	100 000	1	4.855005 seconds
Simulation 2	100 000	2	2.511355 seconds
Simulation 3	100 000	4	1.896803 seconds
Simulation 4	100 000	8	1.090002 seconds

Table 4: Strong scaling of `nbody-bh.c` on the MODI System with 32-core CPUs

`nbody-bh.c` runs very efficiently compared to `nbody.c`. we chose a dataset large enough, so that it would not execute too fast for any meaningful interpretation of the runtime variations as the number of threads increases. Also, by using the same input size it makes it easier to compare the two implementations. You can see that as the number of threads used to execute increases, the simulation time diminishes "almost" linearly. This signifies good use of parallel loops and good strong scaling.

2. b) Find a workload (changing either the number of particles or θ or both) where `nbody.c` is faster than `nbody-bh`. Explain why.

	time in seconds	θ
nbody	12.923906	NA
nbody-bh	11.932377	0.27
nbody-bh	12.859461	0.26
nbody-bh	13.966680	0.25

Table 5: Each simulation is performed on Lenovo Yoga Slim 7x Snap-12C/16/512 for 100 steps, on 10 000 particles using 4 threads measured in simulation time

`nbody.c` is faster than `nbody-bh.c` when $\theta < 0.26$. This could be explained via the fact that the Barnes-Hut algorithm acts as an approximation for the naive algorithm, where the θ parameter determines how close this approximation is. When $\theta = 0$ the Barnes Hut algorithm is equivalent to the naive algorithm. When θ becomes sufficiently small, the overhead of constructing the oct-tree which the Barnes-Hut algorithm uses, outweighs the benefit of approximating the calculations, at least when it comes to how long it takes to execute the simulations, as a smaller θ also makes the simulation more accurate, which in some cases is more important than how long

it takes to simulate. For the variables stated in the table above, this happens when $\theta < 26$ approximately.

3. c) Do you have any memory leaks, and how can you know?

Similarly, we tested the 'nbody-bh.c' simulation for memory leaks using the same method. The 'leaks' command was run with the following:

Command used:

```
leaks --atExit -- ./nbody-bh particles particles.out_bh warnings_out_bh 10
```

The output again confirmed that no memory leaks were found during the execution of the 'nbody-bh' simulation:

```
Process 60337: 204 nodes malloced for 24 KB
Process 60337: 0 leaks for 0 total leaked bytes.
```

This result indicates that, just like in the 'nbody' simulation, all memory allocated during the simulation was properly freed, and there were no memory leaks.

During the implementation of the octree structure, a segmentation fault occurred when calculating the corner positions of the child nodes. The issue arose due to a division by 2 in the following code:

```
bh->children[i]->corner.x = bh->corner.x + ox *bh->l / 2;
bh->children[i]->corner.y = bh->corner.y + oy * bh->l / 2;
bh->children[i]->corner.z = bh->corner.z + oz * bh->l / 2;
```

The division by 2 was originally introduced to adjust the position of child nodes in relation to their parent. However, this caused a segmentation fault because the logic was improperly applied. Specifically, the division by 2 resulted in incorrect memory addressing when accessing the child node structures, leading to invalid memory access and triggering the segmentation fault.